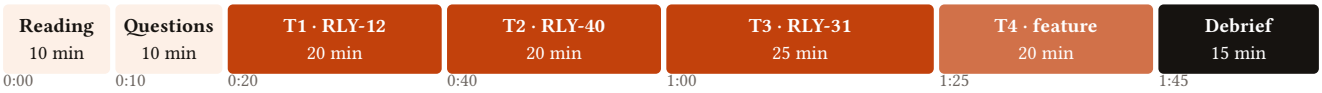
INTERVIEWER GUIDE

AI Session: Relay

The session at a glance

Two hours, six blocks. Keep this page next to you; everything else in this guide is detail for one of these rows.



Clock	Block	You do	Free periods
−0:15	Before	On the candidate's machine: `go run ./cmd/cluster` in one terminal, `bin/relayctl status` in another; nine cells `active w1 v0 up`. Start your own copy on your laptop to check claims without touching theirs. Note their AI tool, skills, MCP servers and rules file; ask them to turn web and GitHub access off. Cards in order.	—
0:00	Welcome	Read "What you hand to the candidate" aloud. Hand the reading card. Start the timer.	—
0:00–0:10	Reading	Watch. Note what they open themselves, what they ask the AI, and whether they start the cluster.	**History**
0:10–0:20	Questions	AI closed, editor open. Ask 1, 2, one flow question (11 or 12) and three or four others. Note answers; correct nothing.	none: you are asking
0:20–0:40	Task 1	Hand the RLY-12 card. Note whether they reproduce before they fix. At 0:35: "five minutes, then tag".	**Projects**
0:40–1:00	Task 2	Hand the RLY-40 card (the memory leak). At 0:55: "five minutes, then tag".	**Way of working**
1:00–1:25	Task 3	Hand the RLY-31 card (the incident). At 1:20: "five minutes; write the incident note now".	**AI setup**
1:25–1:45	Task 4	Hand the feature card you chose. Skip this block if you are behind; the debrief gets the time.	any topic left
1:45–2:00	Debrief	AI closed. Always questions 2 and 6; then as many others as fit.	none
after	Wrap up	Collect the repository with tags, the AI transcript and the recording. Fill in the ratings the same day, from the bug sheet and your notes.	—

Free periods are the minutes when the candidate is waiting for the AI to finish. Use them for the conversation topic of that block (see "While the AI works"), and stop as soon as the output arrives. A block with no waits gets no conversation; that is fine.

If a task ends early, ask its one follow-up probe (in "While the AI works"), then hand the next card. Spare minutes go to the next task, never to the debrief. **If you run late,** stop the task at its time box anyway and have them tag; drop Task 4 first; never give Task 3 less than 20 minutes or the debrief less than 10.

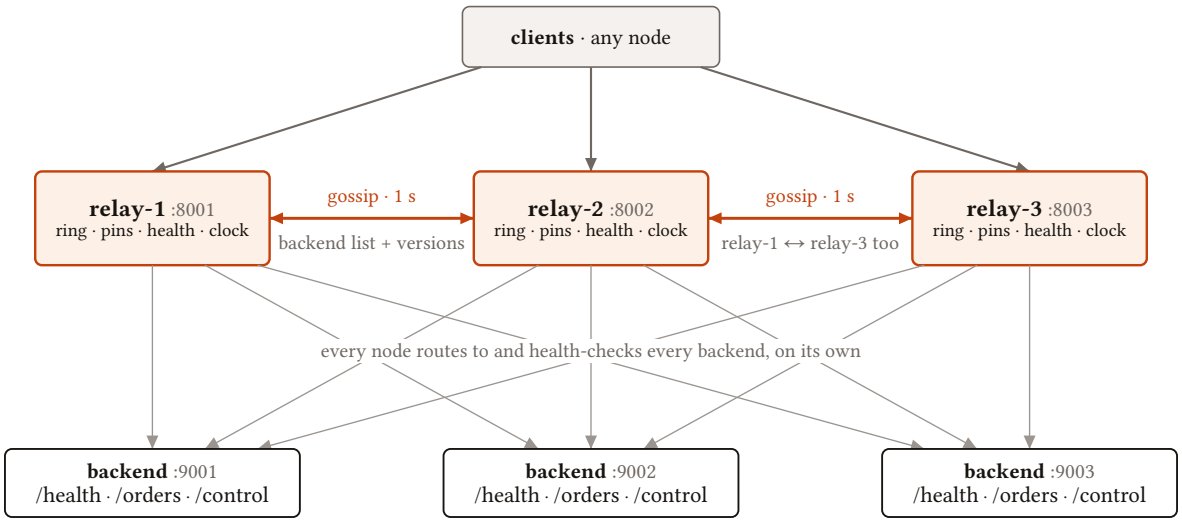
What this session measures

A two-hour session on a codebase the candidate has never seen, with an AI assistant of their choice. They read the code with the assistant and answer our questions about its packages, its functions and its flow. Then they work three tickets from operations, each a symptom whose cause we do not tell them, and, time allowing, build a feature you pick from a menu.

Assume the AI finds the bugs. We tested this codebase against two models with the plain prompt “find all the bugs”. A mid-tier model that only read the code found five of the seven planted bugs in two minutes; it missed the hash skew, which only a measurement shows, and saw the pin split only from the side. A strong model that was allowed to start the cluster found all seven in about eight minutes, among 22 findings: the seven, six real problems we did not plant, design gaps, and small issues, with the incident’s most expensive bug ranked “medium”. So the question is not whether the candidate’s AI can find a bug. It is what the candidate does with a long, confident, mixed list: whether they reproduce before they fix, rank by the ticket in front of them, keep each fix to its cause, and notice when the list is wrong.

Why this codebase

Relay is a distributed HTTP load balancer in Go, standard library only: about 3,000 lines of code and 1,000 lines of tests, plus two tools that make it easy to run. Several nodes share a versioned backend list by gossip, route with a consistent-hash ring, pin sessions to the backend that holds them, check backend health on their own, and retry what is safe to retry. It has configuration, metrics, an admin API with a token, and the Go profiler.



Each node is the same program. The only thing they exchange is the backend list, by gossip; health, pins, the ring, the clock and every request’s retry state stay inside one node.

The seven planted bugs were chosen against what an AI does well. None sits under a comment that contradicts it, and the README promises nothing precise enough to diff against. Two only show when the code runs and something is measured (B1, B7); two need a sequence in time across nodes (B3, B5); one hides behind a half-true comment that sounds like real net/http knowledge (B4); one is a JSON zero value that crosses three files (B6); one is a classic the AI usually knows (B2). All seven were reproduced on a running cluster and with failing tests; the numbers below come from those runs.

	Bug	Where	Effect	Surfaces in
B1	The ring hash is 32-bit FNV-1a over addr-i	ring · hash	Three equal backends get 18%, 50% and 32% of the keys	Task 3, F5
B2	X-Forwarded-For is read from the left	proxy · ClientIP	Behind a trusted proxy, a client writes its own key and picks its backend	Reading, debrief
B3	Tombstones expire, and a restarted node re-seeds the backend	cluster · Sweep, NewState	A removed backend comes back on every node after any node restarts	Task 1 (reported)
B4	EOF and connection resets count as “never sent”	proxy · notSent	A POST the backend handled before dropping the connection is sent again: duplicate orders	Task 3
B5	Pins are per node, and a failover re-pins	node · pick	After a failover, one node keeps a user on the fallback for good while the others send them home: sessions split	Task 3, F3
B6	A backend added without a weight gets weight 0	cluster · Apply	Shown as “active” and on the ring on every node, serves nothing	Task 1 (reported)

	Bug	Where	Effect	Surfaces in
B7	Gossip builds a new http.Transport for every exchange	cluster · Gossiper.client	Every node leaks about 6 goroutines and 3 MB a minute, with no traffic at all	Task 2 (reported)

Format

Individual, about 120 minutes a reading phase, three tickets, an optional feature, and a 15 minute debrief. Book a two-hour slot.

Read first, then questions the session opens with 10 minutes of reading the code with the AI, then 10 minutes of questions about its packages, its functions and the flow of a request, with the AI closed. Nobody changes code before the questions are done.

Talk while the AI works when the candidate is waiting for the AI, use the time to talk about their history, their projects and their way of working. Stop as soon as the output arrives. See "While the AI works" below.

Their own AI any tool is allowed; Claude Code, Cursor, Copilot, or a plain chat window. Note which one it was and whether it could run commands by itself. Note before the session which skills, MCP servers and rules files (CLAUDE.md, AGENTS.md, Cursor rules) are configured. This repository is public and contains this guide: ask the candidate to turn web and GitHub access off for the session, or note that it stayed on.

Go only there is one implementation. A candidate who does not write Go can still take the session, since the code is plain, but say so on the sheet; the Coding rating then weighs judgment over idiom.

Observed one interviewer with a notes sheet, plus screen recording. Without an observer, the AI transcript and the git history keep most of what you need.

Commit after every task the candidate tags the end of each task (task-1, task-2, …). Unfinished work is fine and is evidence in itself.

Tickets are handed out one at a time Task 3 is easier for someone who already looked at pins, and an AI audit run early will mention every bug at once. What the candidate does with that is evidence; handing out the tickets early spoils it.

Preparation

- ☐ Copy ai-session/relay/ from this repository into a **new, private** repository with a neutral name (for example relay) and a single initial commit. Do not link to this repository: it is public and contains this guide.
- ☐ Do **not** fix anything. Every ticket depends on the bugs being there.
- ☐ Go 1.27 or later; nothing else. Ports 8001 to 8003 and 9001 to 9003 must be free; if not, go run ./cmd/cluster -node-port 18001 -backend-port 19001, and relayctl follows (it reads logs/cluster.json). B1's numbers depend on the addresses; on other ports, measure them again.
- ☐ Confirm the setup: go test -race ./... passes; go run ./cmd/cluster prints "cluster up"; bin/relayctl status shows nine cells active w1 v0 up; bin/relayctl send -n 3000 serves about 529 / 1506 / 965. Type quit and start the cluster again just before the candidate arrives: state lives in memory, so a fresh start is a clean slate.
- ☐ The cluster tool shortens the tombstone TTL to 30 seconds and trusts X-Forwarded-For from 127.0.0.1, so RLY-12 and B2 can be shown in a minute. Keep those defaults.
- ☐ Choose the feature for Task 4 from the menu, according to the level you are hiring for.
- ☐ Print this guide and write each brief on a separate card.

What you hand to the candidate

> **Read verbatim:** You are joining the team that maintains Relay, a distributed load balancer: several nodes that share a list of backends and route requests to them. You have never seen this code. You have about two hours and an AI assistant of your choice.
>
> 1. Use the AI as much or as little as you like. We care about how you work, not about whether you use it.
> 2. Think aloud. Tell us what you ask the AI and why, and what you conclude after it answers.
> 3. After each task, commit and tag (git tag task-N). Unfinished is fine.
> 4. If something in a task is unclear, you may ask us. We may or may not answer.
> 5. Keep your AI transcript. We collect it at the end.
> 6. While your AI is generating code, we may talk about your background and how you work. Go back to the output whenever you want; you do not need to finish the sentence.

While the AI works

An agentic tool often works for a minute or more before it shows anything. Do not spend that time in silence: talk with the candidate about their history, their projects and their way of working. It fills dead time, and it gives us a second view of the person next to the one the tasks give.

Rules:

- Talk only while the candidate is **waiting**. Never interrupt them while they are reading output, writing a prompt or thinking aloud about the code.
- Stop as soon as the output arrives, even mid-answer. What the candidate does next is evidence: do they turn to the output and read it, or keep talking and accept it unread? Note which.
- Do not talk about the task, the bugs or the code. That is what the debrief is for, and a hint here spoils the tasks.
- The time boxes do not stop for the conversation. Keep the questions short, and pick up a topic again at the next wait instead of finishing it.
- Ask only about work. No questions about age, family, health, religion or anything else that has no place in an interview.

Each block has its own topic, so the conversation moves through the candidate's story in order and you never have to think of a question on the spot. Go down the list; tick each question you asked; when a topic runs out before the block does, borrow from the next one.

Block	Topic	Questions, in order	Write down
Reading 0:00–0:10	**History**	1. How did you get into software? 2. Which job or project taught you the most, and what? 3. What bug did you chase the longest, and how did you find it in the end?	How they explain a past problem: cause, or story?
Task 1 0:20–0:40	**Projects**	1. Tell us about a system you work on now. What is your part of it? 2. Has it run behind a load balancer or with service discovery? What went wrong with it? 3. Which decision in it would you make differently today? 4. When were you last on call, and what was the incident?	Can they draw their own system in words? Do they know what is distributed in it?
Task 2 0:40–1:00	**Way of working**	1. How do you use an AI assistant day to day, and what do you not use it for? 2. When did an AI last give you something wrong, and how did you notice? 3. How do you review a teammate's code? What do you look at first? 4. How have you chased a memory problem before, and with what tools? 5. When do you write tests: before, during or after?	Anything the session contradicts, such as "I always read the diff" followed by an unread accept.
Task 3 1:00–1:25	**AI setup**	1. Which skills and MCP servers have you configured, and why those? 2. Is there one you wrote yourself? Show it. 3. Which did you try and remove, and why? 4. What is in your rules file? 5. Which of these did your AI use today, and did it help?	Whether they know what their setup does, not how big it is.

A setup is not scored by its size. A candidate with a plain tool and no configuration loses nothing. What counts is whether they know what their setup does and why it is there.

Early finish, one probe per task, then hand the next card:

Task 1 "relay-3 is switched off during the removal and comes back an hour later. Show me that it ends up right." (In the cluster terminal: `stop relay-3`, then `start relay-3`.)

Task 2 "How would we have caught this before production?" (A goroutine gauge with an alert, or a test like the reference one.)

Task 3 "Write the one test that would have caught the uneven spread." (Equal weights on the cluster's own addresses, each share within a band.)

Score this conversation in the same four competencies as everything else; do not give it a score of its own. Most of it is evidence for **effective communication**.

READING **Read the code** 20 minutes

BRIEF FOR THE CANDIDATE

Read the code with your AI assistant, in any way you like: ask it for an overview, go through the files together, run the tests, start the cluster with `go run ./cmd/cluster` and poke it with `bin/relayctl` (see the README). Do not change any code. You have 10 minutes.

Then we close the AI and ask you about the packages, the functions, and how a request flows through them. The code stays open, and you may look at it while you answer.

No trap. This is the baseline for how the candidate reads with the AI, and the questions show what they took away from it: their own picture of the code, or the assistant's summary. For the questions, the AI window is closed or minimised; the editor stays open. Finding the answer in the code is a good answer. Guessing from what the AI said, without looking, is the thing we want to see.

Keep the two pictures below in front of you while you ask. Each step names where it happens in the code; the tags show where a bug lives, so you can tell which answers "touch" one.

A request (question 11)

1. `node.Handler` /relay/… is the node's own API; anything else goes to the proxy
2. `proxy.ServeHTTP` request id; reads the body (up to 1 MiB) and keeps it for a retry
3. `proxy.ClientIP` key = X-Relay-Key, or the client's address; behind a trusted proxy, from X-Forwarded-For — B2
4. `node.pick` the key's pin if its backend is usable; otherwise the ring, and pin the answer — B5
5. `ring.Lookup` hash the key, walk clockwise to the first usable backend — B1
6. `proxy.forward` new request, its own timeout, X-Forwarded-For appended; the answer read in full
7. `proxy.shouldRetry` client gone: no; error: if idempotent or never sent; 502–504: if idempotent — B4
8. `proxy.reply` status, headers and body back, plus X-Relay-Backend and X-Request-Id

A change, gossip and health (questions 6, 7, 8, 12)

1. `node.putBackend` PUT → State.Apply: a partial change; an unknown or removed backend starts from a fresh entry — B6
2. `State.stamp` clock + 1; the entry gets that version, this node as origin, and the time
3. `node.rebuild` new ring from the active entries; pins to backends that left are dropped
4. `Gossiper.Round` every second, exchange with each peer through client() — B7
5. `State.Merge` clock moves past every version; the newer entry wins, origin breaks ties
6. `State.Sweep` tombstones older than the TTL are forgotten; a restarted node has its seed list at version 0 — B3
7. `health.Record` each node, every second: down after 3 failures in a row, up after 2 successes

Ask six or seven of the questions below: always 1 and 2, and at least one flow question (11 or 12). After each answer ask "where is that?" and let them point at the code. Do **not** say whether an answer is right, and do not correct it. If they name a bug now, write it down and move on.

	Question	What a good answer says
1	What are the packages, and what does each one do?	`config` reads defaults, a file and flags; `ring` places backends on the hash ring; `affinity` keeps session pins; `health` checks backends; `cluster` keeps the versioned list and gossips it; `proxy` forwards, finds the client address and retries; `metrics` counts; `node` wires them and serves

	Question	What a good answer says
		the admin API; under cmd, the node, the demo backend and the two tools.
2	What do the nodes share, and what does each keep to itself?	Shared by gossip: the backend list (status, weight, version, origin, time). Local: health, pins, the ring (built from the list, so equal only when the lists agree), the clock, metrics. A candidate who adds "so two nodes can route the same key differently, through health or pins" has seen B5′s ground.
3	How does a request choose its backend?	pick: the pinned backend if it is usable, refreshing the pin; otherwise ring.Lookup with the same usable (healthy and not tried), and the answer is pinned. **Touches B5:** a retry or a failover pins the fallback.
4	Behind a trusted proxy, which address is the client's?	ClientIP takes the first parseable entry of X-Forwarded-For. Only a candidate who knows how proxies append adds that the first entry is the one the client wrote. **Touches B2.**
5	When is a request sent to a second backend?	shouldRetry: never if the client is gone; after an error, if the method is idempotent or notSent says it never arrived (a refused dial, EOF, a reset); after 502, 503 or 504, only if idempotent. **Touches B4;** do not push on whether EOF really means "never arrived".
6	How does a change get its version, and why does a change made anywhere win?	stamp increments the clock; Merge moves the clock past every version it receives, so the next local change is higher than anything seen; equal versions are settled by origin.
7	What happens to a removed backend over time?	Remove writes a tombstone with a version and a time; it gossips like any entry; after the TTL Sweep forgets it and Merge refuses expired ones. **Touches B3:** a node started later has the seed list at version 0.
8	PUT /relay/backends/127.0.0.1:9004 with {"status":"active"}: what entry results?	A new entry, active, with the next version, and weight 0, since nothing sets one. Only a candidate who followed Weight into ring.New says it gets no points. **Touches B6.**
9	How is the ring built?	64 points per unit of weight, named addr-i, hashed with 32-bit FNV-1a and sorted; a key goes to the first point at or after its hash. Nobody can tell B1 from the code alone; do not expect it.
10	When is a backend marked down, and is that shared?	Three failed checks in a row, up after two successes in a row, by Record; never gossiped; a backend never checked counts as healthy.
11	Flow: follow curl -H 'X-Relay-Key: alice' localhost:8001/hello to the backend and back.	See the left picture above. Good answers name pick, the pin, Lookup, the per-attempt timeout, and that the whole answer is read before replying.
12	Flow: what happens every second on every node?	The health checker probes every known backend; the gossiper exchanges the whole list with each peer through client(), merges both ways, then sweeps old tombstones. **Touches B7;** the new client per exchange is visible here, and nobody should be expected to see the leak in it.

What to watch for:

- While reading: do they only ask the AI for a summary, or do they open files, run the tests, and start the cluster?
- Specific answers (a file, a function, a field) or general ones ("it uses consistent hashing")? Do they look before answering?
- Wrong answers that repeat a claim the AI made are the main evidence here. Note each one.
- "I don't know, let me look" is a good answer. A confident guess is not.

TASK 1 The backend that came back (two symptoms)

20 minutes

BRIEF FOR THE CANDIDATE

Ticket **RLY-12**, from operations:

“On Monday we removed `127.0.0.1:9003` for re-imaging with `relayctl remove relay-1 9003`. All three nodes showed it as removed. On Tuesday morning relay-2 was restarted by a routine deploy, and 9003 was back in rotation on every node; requests to it failed until we noticed. After the re-image we added it back with `relayctl activate relay-1 9003`. Every node shows it active, and it takes no traffic at all.”

Fix both, with regression tests that fail before the fix and pass after. A removed backend must stay removed through the restart of any node; a backend added back must take its share.

```
bin/relayctl remove relay-1 9003     # all three: removed v1
sleep 35                             # the cluster tool's tombstone TTL is 30s
                                     # in the cluster terminal: restart relay-2
bin/relayctl status                  # :9003 active w1 v0 up, on all three
bin/relayctl remove relay-1 9003
bin/relayctl activate relay-1 9003   # all three: active w0 v.. up, ring lists :9003
bin/relayctl send -n 3000 -node all  # :9003 serves none
```

Trap · B3, a tombstone has to outlive every way back. Tombstones are forgotten after the TTL, which is reasonable for a backend nobody will mention again. But every node’s seed list mentions it: a node that restarts after the TTL holds 9003 as active at version 0, its peers no longer have anything to compare it with, and the old entry wins everywhere. Fixes that work: keep tombstones of seed backends for good (the reference fix; a tombstone is a hundred bytes); or have a restarting node take the list from a peer before it trusts its seed. Taking 9003 out of the seed is the right operational step and not a fix: the next removed seed backend does it again.

Fixes that do not: a longer TTL (later, same failure); making `Merge` ignore version-0 entries (the restarted node still routes to 9003 itself, and its seeds never reach the others, so the nodes disagree); dropping the TTL logic without thinking about growth. The reference fix makes `TestSweepForgetsOldTombstones` fail, because it removes a seed backend: the right answer is to change the test to a backend that was added later, not to delete it.

Trap · B6, everything says it is fine. `Apply` starts a new or re-added backend from a fresh entry and only copies the fields the request carries; `relayctl activate` sends only the status, so the weight is 0, and `ring.New` gives it 0 × 64 points. The ring’s member list still shows it, so `/relay/status` says “active, on the ring” on every node; only traffic shows the truth. Defaulting the weight to 1 in `Apply` is the reference fix; rejecting a new backend without a weight is also fine if stated. Making `ring.New` treat 0 as 1 hides it in one place while gossip keeps carrying 0.

What to watch for:

- Do they reproduce both symptoms on the cluster before asking the AI to fix them? The steps take two minutes.
- B6 is invisible in `/relay/status`. Does the candidate check traffic (`send`) or trust the status?
- Does the B3 regression test include a restart, that is, a new `State` built from the seed after the TTL? A test that only checks “the tombstone spreads” passes on the buggy code.
- Strong candidates ask what a re-added backend’s weight should be: 1, or the weight it had before it was removed.

TASK 2 Memory leak reported

20 minutes

BRIEF FOR THE CANDIDATE

Ticket **RLY-40**, “memory leak reported”:

“relay-2 was OOM-killed on Sunday after nine days up. The memory graph of all three nodes is a straight line up, the same slope every hour, nights and weekends included; a restart resets it. Traffic does not seem to matter.”

Find the leak, fix it, and show us that it is fixed.

What an idle cluster does, measured on our machine; every node the same:

```
              start   after 60s   after 120s
goroutines       51         411          771
heap MB         1.3         4.2          8.2

/relay/debug/pprof/goroutine?debug=1 after two minutes:
  255  net/http.(*persistConn).readLoop
  255  net/http.(*persistConn).writeLoop
  252  net/http server connections
TCP connections established on :8001: 505
```

Trap · the leak is in the quiet part of the code. `Gossiper.client` builds a new `http.Client` with a new `http.Transport` for every exchange, “so a stuck peer costs at most one timeout”. Each exchange leaves a kept-alive connection idle in a transport nobody holds; a hand-built transport has no idle timeout and the server none either, so the connection, its two client goroutines and one server goroutine live for ever. Two peers, once a second: about six goroutines a second per node, with no traffic.

The ticket says traffic does not matter, and `relayctl status` shows zero pins on an idle cluster. Both rule out the AI’s favourite suspects: unbounded pins, metric labels, per-request allocations. The goroutine count climbing on an idle cluster, and a goroutine profile full of `persistConn`, point at the gossip client in two steps.

Fixes: one client per gossiper (the reference fix), with an idle timeout; or `CloseIdleConnections` after each exchange, or keep-alives off, both of which work and give up connection reuse. Setting only the server’s `IdleTimeout` bounds the leak instead of removing it.

What to watch for:

- Do they measure before they guess? `relayctl status` shows goroutines and heap on every node; `/relay/debug/pprof/` is in the README.
- The AI’s first list usually includes the pins map. Pins are zero here. Does the candidate check that before fixing pins?
- “Show us that it is fixed”: a number that stays flat for a minute, or a test that runs fifty rounds and counts goroutines. “The code looks right now” is not a showing.
- Do they leave the per-exchange timeout behaviour intact? The comment’s goal (a stuck peer costs one timeout) is reasonable; `Client.Timeout` on a shared client keeps it.

TASK 3 Tuesday's incident (three symptoms)

25 minutes

BRIEF FOR THE CANDIDATE

Ticket **RLY-31**, an incident report:

- At 14:02 a bad build went out on backend-2 (127.0.0.1:9002). Under load its process crashed every few requests, dropping the connection, until the rollback at 14:08.
- The payments team found 41 orders recorded twice in that window. In every pair, one order is on backend-2 and one on another backend, same customer, same cart, within a second. Each customer saw one success.
- Since Tuesday, support has a steady stream of “logged out at random”. Sessions live in the backend’s memory, and our DNS spreads clients over the three relay nodes.
- Not urgent: backend-2 has run at about three times the CPU of backend-1 for months, and all weights are 1.

Find out what went wrong, fix what is broken in Relay with regression tests, and write two or three sentences for the incident report on what the load balancer did wrong.

```
bin/relayctl orders reset; bin/relayctl fail 9002 crash 0.5
bin/relayctl send -n 30 -method POST -body '{"cart":1}' /orders   # 200 x30
bin/relayctl orders                                  # 40 orders: ten keys twice
bin/relayctl heal 9002
                          # quit and start the cluster again: the retries above moved pins
bin/relayctl send -n 300 -node all                # each node: :9001 76  :9002 113  :9003 111
bin/relayctl fail 9002 health; sleep 4
bin/relayctl send -n 300 -node relay-1              # :9001 84  :9003 216
bin/relayctl heal 9002; sleep 3
bin/relayctl send -n 300 -node all                # relay-1: :9002 none; relay-2 and -3: :9002 113

bin/relayctl send -n 3000                       # :9001 529  :9002 1506  :9003 965
```

Trap · B4, a half-true comment. notSent counts EOF and connection resets as “the backend never received the request”, and explains why: a kept-alive connection the backend had already closed. That case is real, and Go’s transport already retries it by itself for requests it can replay. An EOF that reaches the caller after the request was written means the backend may have done the work, which is exactly what a crashing process does. Only a refused dial proves nothing was sent. The reference fix keeps notSent to dial errors; the test TestPostIsRetriedWhenTheConnectionIsRefused still passes, which is the evidence that safe retries survive. Turning retries off entirely makes it fail. “The service should take idempotency keys” is good advice and not the answer: Relay sent the same POST twice.

Trap · B5, an AI calls it a design gap. Pins and health are per node. During the incident every retried request and every failover re-pinned its key on the node that handled it, and a pin refreshed by every request never expires. After the rollback, the nodes that saw the failover keep those users on the fallback backend for good; the others send them home. A user whose requests alternate between nodes alternates between backends, and loses their session each time. Our strong-model audit listed this as “pins are per node, a design gap, medium”. In this ticket it is the most expensive bug.

The reference fix pins only the ring’s own answer, so a failover is temporary and a key goes home when its backend recovers. It does not make pins agree across nodes after a ring change; that needs shared state or giving pins up, and is a good debrief discussion. Watch for candidates who fix the symptom with a shorter pin TTL.

Trap · B1, only a measurement shows it. 32-bit FNV-1a over names that differ in one character places the points of 127.0.0.1:9001-i and its neighbours in correlated runs: 18%, 50% and 32% of the keys. Nothing in the code looks wrong, and a reading-only audit missed it. The fix is a hash that mixes well (the reference uses the first 8 bytes of SHA-256); a strong candidate adds that changing the hash moves almost every key, so rolling it out node by node splits every session for the length of the rollout, and pins hide part of that.

What to watch for:

- Three symptoms, three causes, 25 minutes. Does the candidate rank them by cost (orders, then sessions, then capacity) or by what the AI listed first?
- Do they reproduce each one? The commands above take a minute each; the AI cannot see the incident, only the code.
- Is the incident note honest and specific (retries after a dropped connection, pins after a failover), or does it blame the bad build alone?

TASK 4 Feature (interviewer's choice)

20 minutes

Pick one feature. The level describes the nature of the work, not the job title. Tell the candidate only the brief. Skip this task if the session is behind.

	Feature	Level	Surfaces
F1	Health path and any 2xx	1	config plumbing
F2	In-flight gauge per backend	1	data races
F3	Graceful drain	2	B5, pins
F4	Retry budget	2	B4, time in tests
F5	Bounded loads	3	B1, pins

F1 Health path and any 2xx

Level 1 · 20 minutes

BRIEF FOR THE CANDIDATE

Make the health check path configurable (`-health-path`, `health_path` in the file) and count any 2xx answer as healthy. Tests included.

What to watch for: Config has three layers (defaults, file, flags); does the new setting reach all three and `Validate`? Today only 200 counts; a 204 from `/health` is a real case.

F2 In-flight gauge

Level 1 · 20 minutes

BRIEF FOR THE CANDIDATE

`/relay/metrics` should show, per backend, how many requests this node has in flight to it right now. Tests included.

What to watch for: The registry has labelled counters and unlabelled gauges; does the candidate extend it or bend a counter? The count must go down on every path, including errors and retries: a `defer` right after the increment. `go test -race` with concurrent requests.

F3 Graceful drain

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

Draining a backend today moves its sessions at once. Make draining stop new sessions only: keys already pinned to it stay until their pins expire, new keys go elsewhere. Tests included.

What to watch for: `rebuild` drops the pins of every backend that left the ring, and a draining backend leaves it. The candidate has to separate “on the ring for new keys” from “allowed for pinned keys”. If B5 is unfixed, a failover during drain moves the key for good anyway.

F4 Retry budget

Level 2 · 20 minutes

BRIEF FOR THE CANDIDATE

A failing backend doubles the load on the others, because every idempotent request is tried twice. Limit retries to 20% of the requests a node handled over the last 10 seconds. Tests included, without sleeping for 10 seconds.

What to watch for: The time has to come from somewhere the test controls. Where does the budget sit relative to `shouldRetry`, and is B4 found on the way?

F5 Bounded loads

Level 3 · 20 minutes

BRIEF FOR THE CANDIDATE

No backend may have more than $\lceil 1.25 \times \text{average} \rceil$ requests in flight on a node, the average taken over the usable backends and counting the new request. A request whose backend is full goes to the next backend on the ring that is not. Keys go to their own backend whenever it is not full. Tests included.

What to watch for: "Consistent hashing with bounded loads" (Mirrokni, Thorup and Zadimoghaddam; HAProxy and Envoy have it). With B1 unfixed, `:9002` is full far more often than the others, which is the first time the skew hurts. And an overflowed request must not be pinned, or B5 makes the overflow permanent.

Debrief (15 minutes)

1. Which state do the nodes share, and which does each keep to itself? Where did local state make two nodes disagree today?
2. Which of the AI's claims turned out to be wrong today? How did you find out?
3. When may a tombstone be dropped? relay-3 was off for a month and comes back: what happens with your fix?
4. You changed (or would change) the ring's hash. How do you roll that out across three nodes without logging everybody out?
5. How would you have caught the leak before production?
6. Your AI listed many problems today. Which did you not act on, and why? (B2 and the extras below are fair answers; "I fixed all of them in one diff" is not.)
7. Which change would you merge with the least confidence?

What the AI will also find

Real problems we did not plant. Our audits found most of them. A candidate who mentions one has read well; one who fixes them all inside a ticket's diff has not kept scope. None is required.

Finding	What to make of it
`/relay/debug/pprof/` is public, cmdline included	True; with `-admin-token` on the command line, the token leaks. Belongs behind the admin token or a separate listener.
Gossip is open when no cluster secret is set	True by design of the defaults; `-cluster-secret` exists. A fair security point.
A change made on a node that restarted a second ago can be reverted	True: the clock starts at 0 until the first gossip round. Narrow, real.
Draining drops a backend's pins at once	True; that is F3.
Pins grow with every new key a client invents	True and bounded by the TTL. Not the RLY-40 leak: pins are zero on an idle cluster.
Whole responses are buffered	A stated assumption in `forward`; breaks streaming.
Health counts only 200 as up	True; that is F1.
`relayctl fail` replaces earlier failure settings	Demo tool only.

Scoring

Score the session with the **same four competencies** and the **same scale** as the evaluation document: **Weak · Borderline · Strong · Not assessed**. Do not invent a per-task score. Each competency gets one rating for the whole

session, backed by the evidence you wrote down; a competency you saw no evidence for is **Not assessed**, not a guess. Write next to the ratings which feature you chose and its level.

Competency	Weak	Borderline	Strong
Data structures, algorithms and problem solving	Fixes what the AI listed in the order it listed; cannot say at the debrief what is shared and what is local, or why the backend came back	Finds the causes with the AI's help but cannot say why EOF does not mean "not sent", or why pins split	Reasons from state and time: restarts, failovers, what each node knows; ranks the incident's causes by cost; explains the hash skew with a measurement
Coding	One large diff for many findings; tests pass before the fix too; a test deleted to make the suite pass	Working fixes, but tests on one node or one state, or `time.Sleep` where a clock belongs	Small diffs per cause; each regression test fails first; restarts and failovers inside the tests; the leak shown fixed with a number
Effective communication	Prompts like "fix the memory leak"; accepts "design gap" or "the service should be idempotent" as the answer	Explains what they are doing but leaves choices implicit (re-added weight, what a failover pins)	Prompts carry the ticket, the commands and the output; challenges the AI with a run; says which findings they are not fixing and why; an incident note that does not shift blame
Speed and getting to a result	Never starts the cluster; applies suggestions until the tests pass	Gets there, but slower than the time boxes because checks come late	Reproduces before fixing, verifies after, and still closes Tasks 1 to 3 in time

Fill in the bug sheet as you go. It is not a score; it is the evidence the ratings point back to.

	Found by	Reproduced?	Root cause fixed?	Test failed before fix?
B1	candidate / AI / no			
B2	candidate / AI / no			
B3	candidate / AI / no			
B4	candidate / AI / no			
B5	candidate / AI / no			
B6	candidate / AI / no			
B7	candidate / AI / no			

Also record three plain counts per task: **files the candidate opened**, **commands they ran against the cluster**, and **AI claims accepted without checking**.

Collecting data

Collect the git history with the `task-N` tags and the final working tree; the AI transcript; the screen recording with the candidate's audio; and the interviewer's sheet with timestamped notes and the bug sheet.

Reference fixes

A short version for the interviewer. With all seven applied, every existing test passes under `-race` except `TestSweepForgetsOldTombstones` (see Task 1), which passes once it removes a non-seed backend; and a regression test for each bug fails before its fix and passes after it.

B1 · a hash that mixes

```
// ring.go: points become uint64
func hash(s string) uint64 {
    sum := sha256.Sum256([]byte(s))
    return binary.BigEndian.Uint64(sum[:8])
}
```

Three equal backends then serve 915, 1031 and 1054 of 3000 keys.

B2 · walk back from our own peer

```
// clientip.go, in ClientIP, after the peer was found trusted
hops := strings.Split(strings.Join(r.Header.Values("X-Forwarded-For"), ","), ",")
client := peer
for i := len(hops) - 1; i >= 0; i-- {
    ip, err := netip.ParseAddr(strings.TrimSpace(hops[i]))
    if err != nil {
        break
    }
    client = ip.Unmap().String()
    if !isTrusted(client, trusted) {
        break
    }
}
return client
```

B3 · keep the tombstones of seed backends

```
// state.go: NewState records the seed in s.seed
func (s *State) expired(e Entry) bool {
    return e.Status == Removed && !s.seed[e.Backend] && s.now().Sub(e.Since) > s.ttl
}
```

B4 · only a refused dial proves nothing was sent

```
func notSent(err error) bool {
    var op *net.OpError
    return errors.As(err, &op) && op.Op == "dial"
}
```

B5 · a failover is not pinned

```
func (n *Node) pick(key string, usable func(string) bool) (string, bool) {
    if b, ok := n.pins.Get(key); ok && usable(b) {
        n.pins.Put(key, b)
        return b, true
    }
    r := n.currentRing()
    owner, _ := r.Lookup(key, nil)
    b, ok := r.Lookup(key, usable)
    if ok && b == owner {
        n.pins.Put(key, b)
    }
    return b, ok
}
```

B6 · a new entry has weight 1

```
// state.go, in Apply
e = Entry{Backend: backend, Status: Active, Weight: 1}
```

B7 · one client per gossiper

```
func (g *Gossiper) client() *http.Client {
    g.clientOnce.Do(func() {
        g.httpClient = &http.Client{
            Timeout: g.Timeout,
            Transport: &http.Transport{
                DialContext:         (&net.Dialer{Timeout: g.Timeout}).DialContext,
                TLSHandshakeTimeout: g.Timeout,
                IdleConnTimeout:     time.Minute,
            },
        }
    })
    return g.httpClient
}
```

The regression test runs fifty rounds against a test peer and fails if more than ten goroutines are left: 6 → 156 before the fix.

Showing each bug on the cluster

For the prep machine, and for checking a candidate's claim during the session on your own copy. Two terminals:

Terminal A `go run ./cmd/cluster`
Stays open. Takes `ps`, `stop relay-3`, `start relay-3`, `restart 9002`, `logs relay-1`, `quit`.

Terminal B `bin/relayctl ...`
Every command below. `bin/relayctl help` lists them all.

State lives in memory; for a clean slate, `quit` in terminal A and start again. `bin/relayctl heal 9002` undoes any `fail`. The commands for B1, B3, B4, B5 and B6 are under Tasks 1 and 3, with the outputs we saw; B7′s numbers are under Task 2. B2:

```
$ for x in 1.1.1.1 2.2.2.2 3.3.3.3 4.4.4.4 5.5.5.5; do
>   curl -s -o /dev/null -w "$x -> %header{x-relay-backend}\n" -H "X-Forwarded-For: $x" localhost:8001/hello
> done
1.1.1.1 -> 127.0.0.1:9002
...
5.5.5.5 -> 127.0.0.1:9003
```

The cluster tool trusts `127.0.0.1`, so `curl` plays the trusted proxy; whatever the client puts first decides the key.

Caveats

- Verified with Go 1.27.3 on Linux, three nodes and three backends on `127.0.0.1`. B1′s shares depend on the addresses; B4′s duplicate count varies from run to run, since the drops are random; B7′s rate is for two peers and one gossip round a second.
- The AI probes were run in November 2026 with the prompt "find all the bugs", once with a mid-tier model reading only and once with a strong model allowed to run the cluster. Newer models will do better. The session does not depend on the AI failing; it depends on what the candidate does with what the AI says.
- The code lives in a public repository. If a candidate could plausibly have seen it, prefer another session.